

Vision Transformers and Multimodal Models

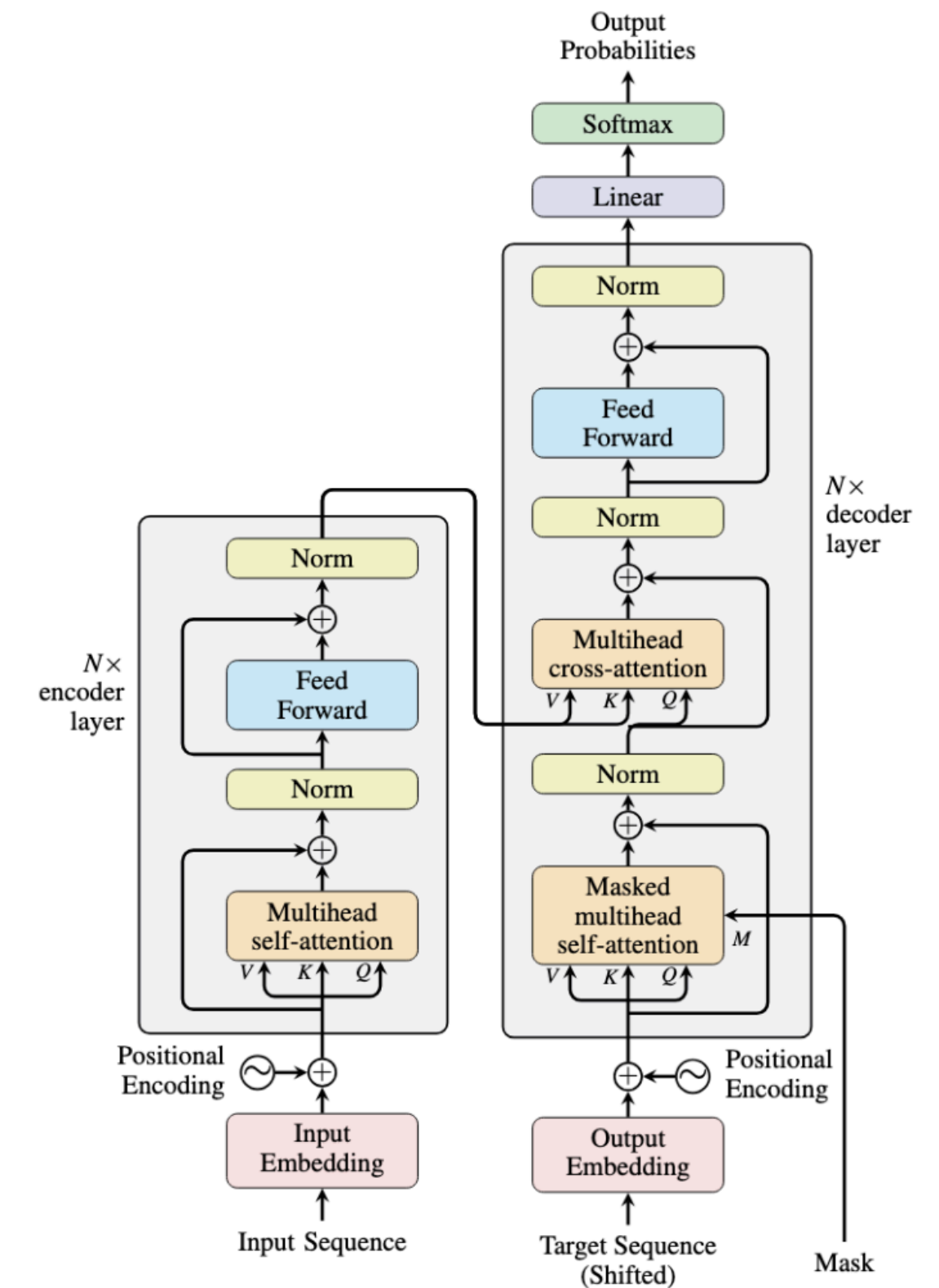
Anna Vettoruzzo

Overview

- Vision transformers (ViT)
- ViT in Python
- Attention visualization
- Multimodal models with vision and text
- Multimodal models with other modalities than vision
- What's next?

Transformer models for text (recap)

- **Tokenization**: text is transformed into a sequence of vectors (token embeddings).
- **Positional encoding**: we represent the position of the tokens by adding an additional vector to each token.
- **Self-attention**: is used to learn interactions between tokens.
- **Causal masking**: we apply a masked self-attention strategy to mask future tokens.



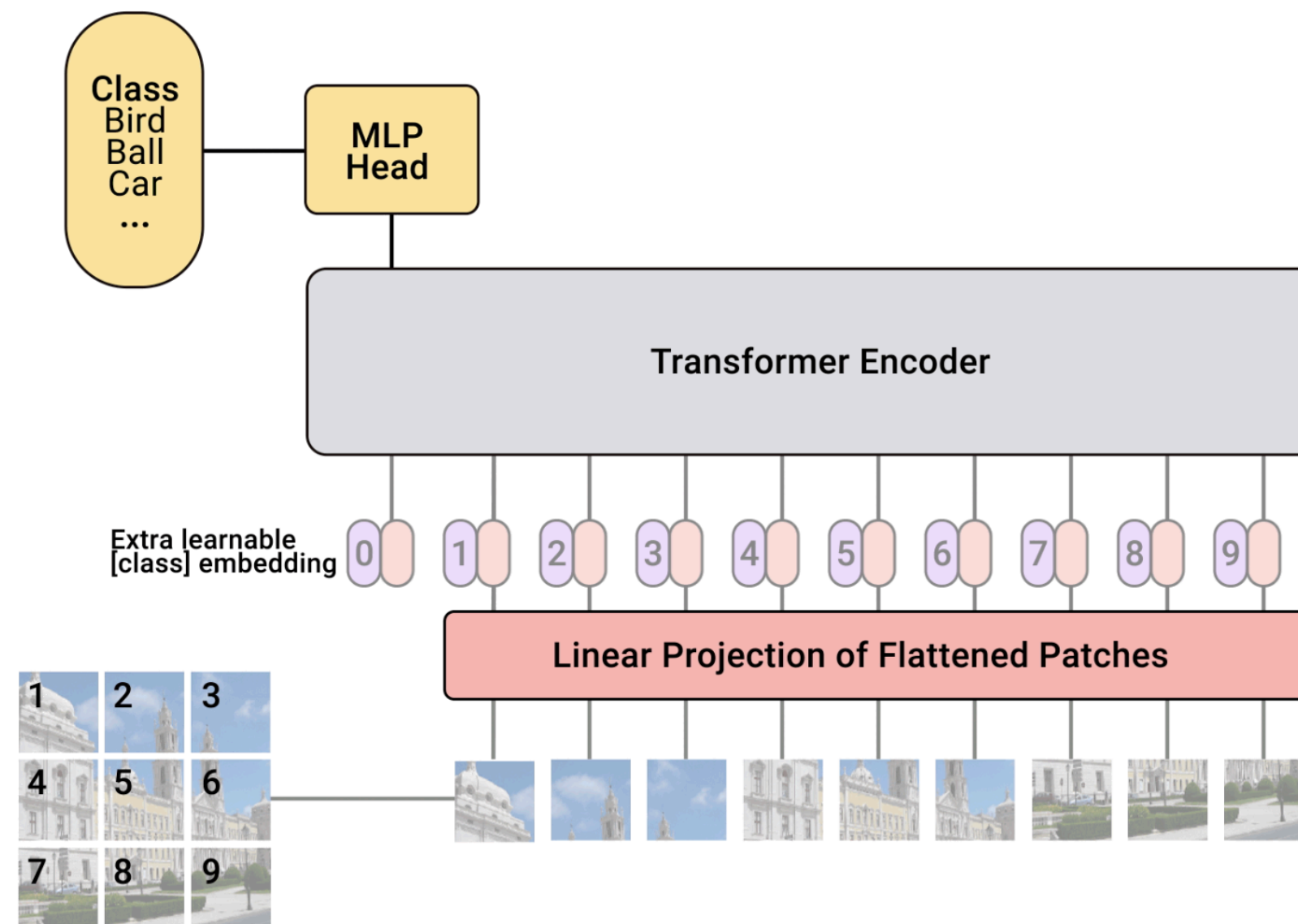
Source: <https://machinelearningmastery.com/encoders-and-decoders-in-transformer-models/>

Vision transformers (ViTs)

We want to transform an image into a sequence of vectors and use an architecture similar to the one used for text.

Steps:

1. Divide an image into patches
2. Flatten the patches into vectors
3. Use a standard transformer (the encoder part)



Python code to implement a ViT

Demonstration

We'll experiment with the [CIFAR-10 dataset](#), which consists of 32x32 color images divided into 10 classes.

- ViTs are quite expensive on large images
- This ViT takes about an hour to train (we'll run it from a checkpoint)

Image 1



Image 2

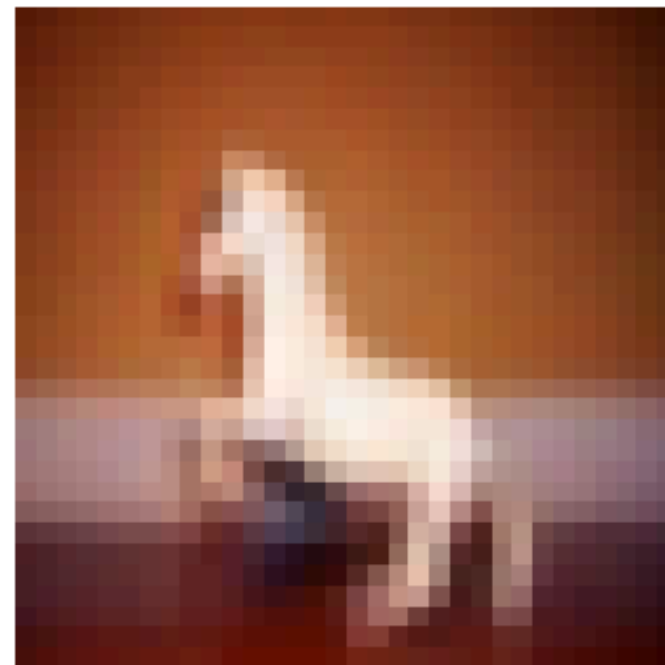


Image 3

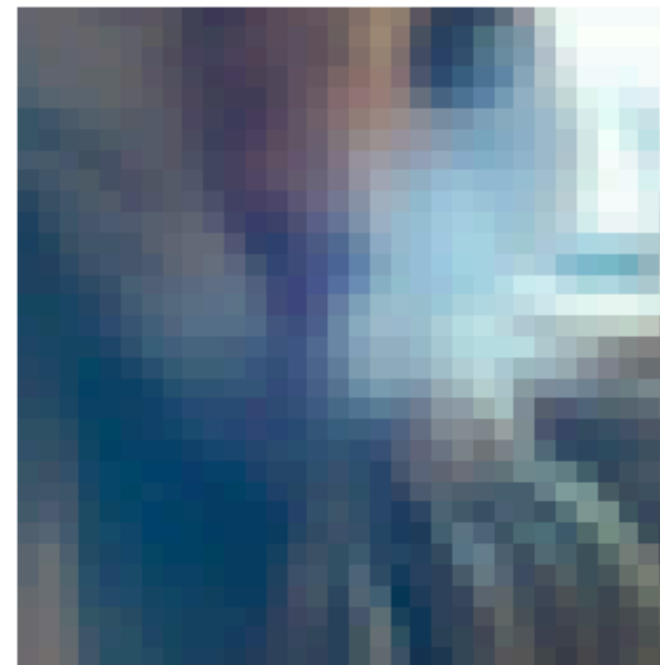
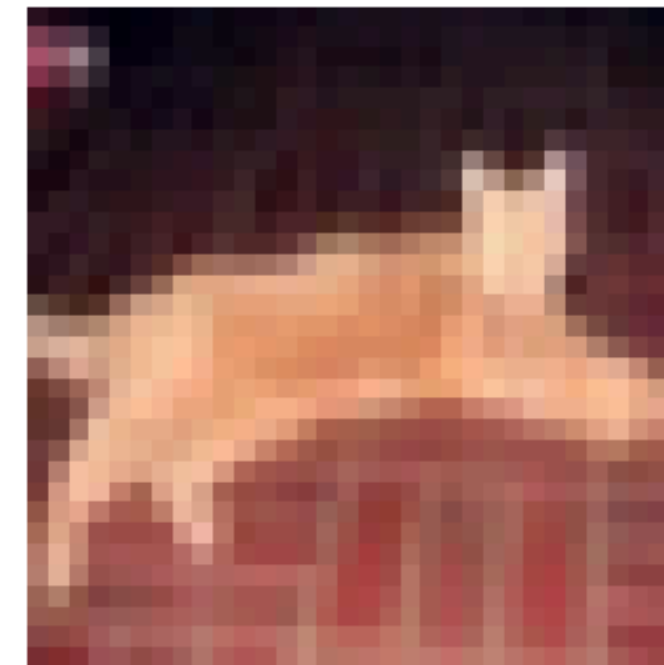


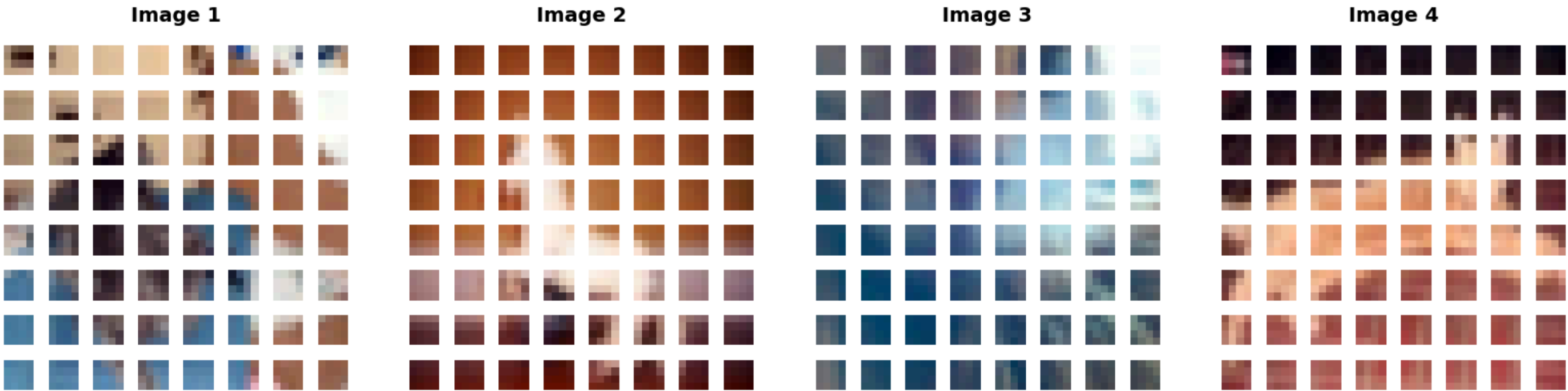
Image 4



Patchify

- Split $N \times N$ image into $(N/M)^2$ patches of size $M \times M$.

```
B, C, H, W = x.shape # Batch size, Channels, Height, Width
x = x.reshape(B, C, H//patch_size, patch_size, W//patch_size, patch_size)
```

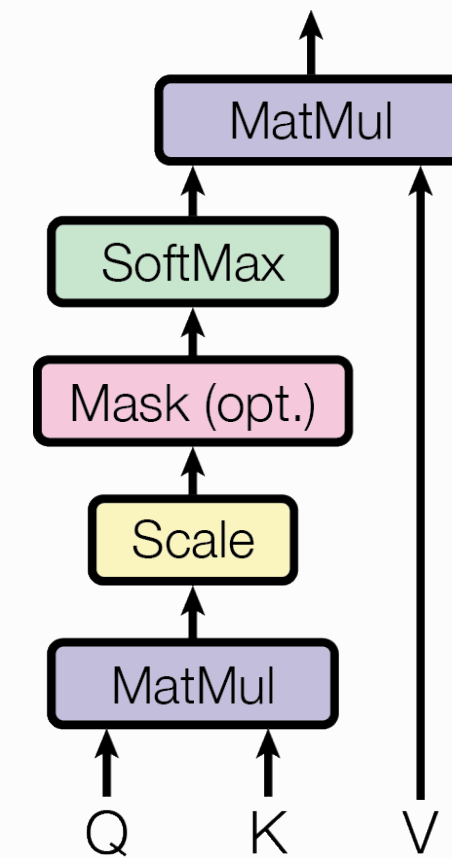


Self-attention

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$

```
def scaled_dot_product(q, k, v):  
    # dot prod  
    attn_logits = torch.matmul(q, k.transpose(-2, -1))  
    # scaling  
    attn_logits = attn_logits / math.sqrt(q.size()[-1])  
    # softmax  
    attention = F.softmax(attn_logits, dim=-1)  
    # dot prod  
    values = torch.matmul(attention, v)  
    return values, attention
```

Scaled Dot-Product Attention



A different perspective for better understanding QKV

QKV is used to mimic a *search-and-match* procedure to find how much two tokens in a sequence are relevant (the weights) and what is the context (the values).

- Query (Q) acts as the current token's question: "What am I looking for?"
- Key (K) is the descriptor for each token: "What information do I have?"
- Value (V) is the actual content or information payload of the token.

Example

We want to search for more information on the self-attention mechanism (a Query) in a library. We ask the librarian whether she can retrieve some information for us. Each book is represented as a Key-Value pair where the Key is the title and the Value is the actual book content. The librarian compares your search Query with the Keys of all the books, measures the similarities and rank the results based on it. She then focuses on the content (Value) of those specific books to give you the answer you need.



Image generated with Nano Banana.

Multi-head attention (simplified)

- We project the input to a high-dimensional space and we divide it into **multiple heads**. This allows different heads to focus on different features.
- We reshape the data by stacking the heads together, so that the self-attention processes all of them simultaneously.
- We concatenate the results from all heads before passing it to a linear layer that project it back to the original input dimension.

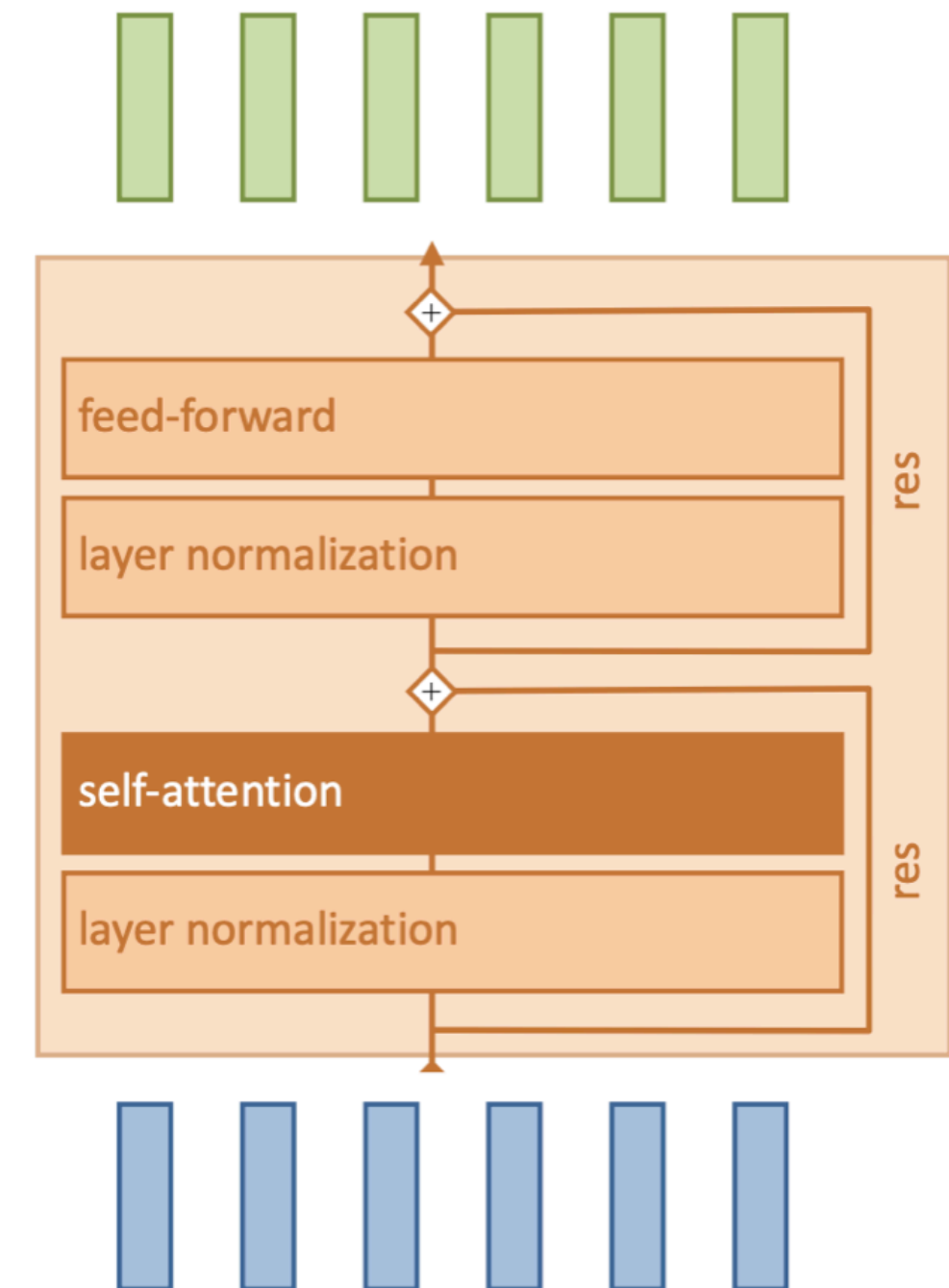
```
qkv = nn.Linear(input_dim, 3*embed_dim)(x) # project to embed_dim
qkv = qkv.reshape(batch_size, seq_length, num_heads, 3*head_dim) # reshape by stacking the heads together
q, k, v = qkv.chunk(3, dim=-1)

values, attention = scaled_dot_product(q, k, v, mask=mask) # self-attention
values = values.reshape(batch_size, seq_length, embed_dim) # concatenate the results from all heads
out = nn.Linear(embed_dim, input_dim) # project back to the original input_dim
```

Attention block

```
def __init__(self, embed_dim, hidden_dim, num_heads, dropout=0.0):
    self.layer_norm_1 = nn.LayerNorm(embed_dim)
    self.attn = nn.MultiheadAttention(embed_dim, num_heads)
    self.layer_norm_2 = nn.LayerNorm(embed_dim)
    self.linear = nn.Sequential( # Feed-forward layer
        nn.Linear(embed_dim, hidden_dim),
        nn.GELU(), nn.Dropout(dropout),
        nn.Linear(hidden_dim, embed_dim),
        nn.Dropout(dropout)
    )
```

```
def forward(self, x):
    inp_x = self.layer_norm_1(x)
    x = x + self.attn(inp_x, inp_x, inp_x)[0] # self-att + res
    x = x + self.linear(self.layer_norm_2(x)) # feed-fw + res
    return x
```

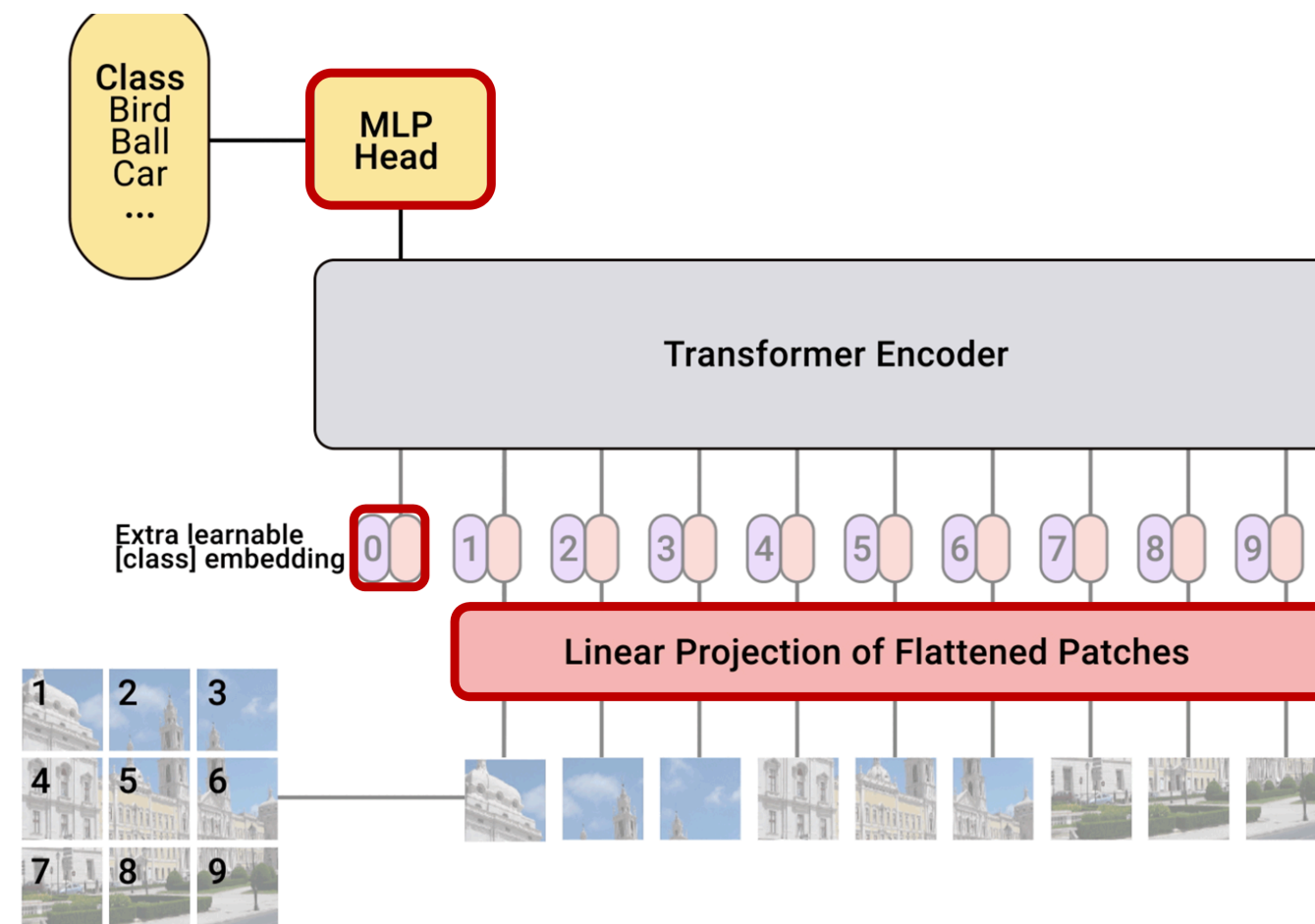


Note: There are usually multiple attention blocks, or layers. At the beginning of the ViT, the tokens will be representative of patches in the input image. However, deeper attention layers will compute attention on tokens that have been modified by preceding layers.

Completing the ViT implementation

Final steps:

- Linear patch embeddings to map image patches to D-dimensional vectors
- Add classification token to the input sequence
- 2D positional encoding so the model understands the spatial layout of the patches
- A small MLP head to map [CLS] token to prediction

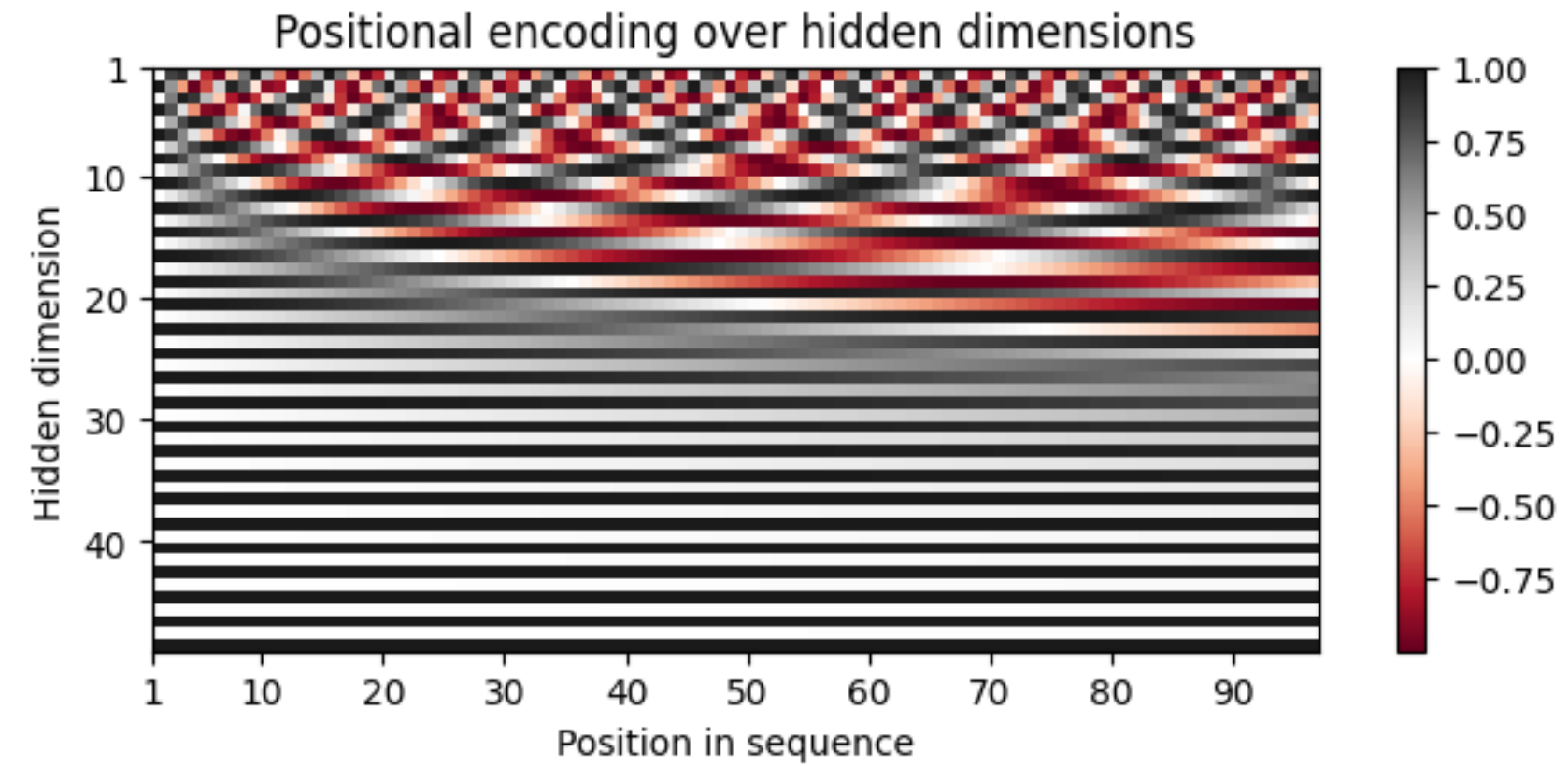


Positional encoding

- We implement this pattern and run it across a 2D grid:

$$PE_{(pos,i)} = \begin{cases} \sin\left(\frac{pos}{10000^{i/d_{\text{model}}}}\right) & \text{if } i \bmod 2 = 0 \\ \cos\left(\frac{pos}{10000^{(i-1)/d_{\text{model}}}}\right) & \text{otherwise} \end{cases}$$

where p is the index of the token in the sequence and i represents the index in the embedding vector, and d_{model} is the embedding dimension of the model.



Why using sinusoidals?

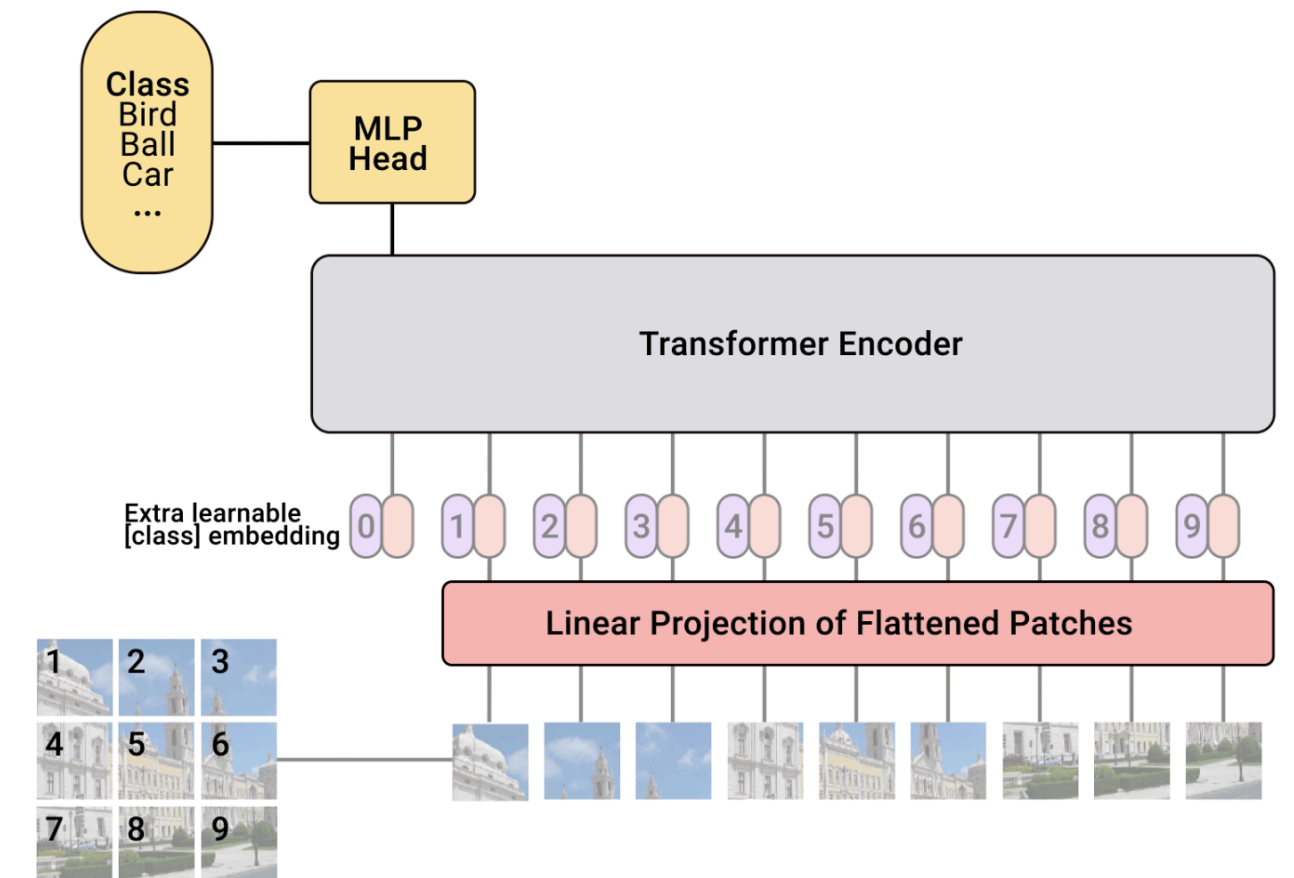
If we use simple integers the values could get massive for long sequences and the model would not be able to generalize to sequence lengths it hasn't seen before.

Sinusoidals have some important characteristics:

- each position in the sequence has a unique encoding
- periodic patterns ensure that similar positions have similar encodings
- easily differentiable, hence suited for backpropagation during training
- allows models to handle any sequence length

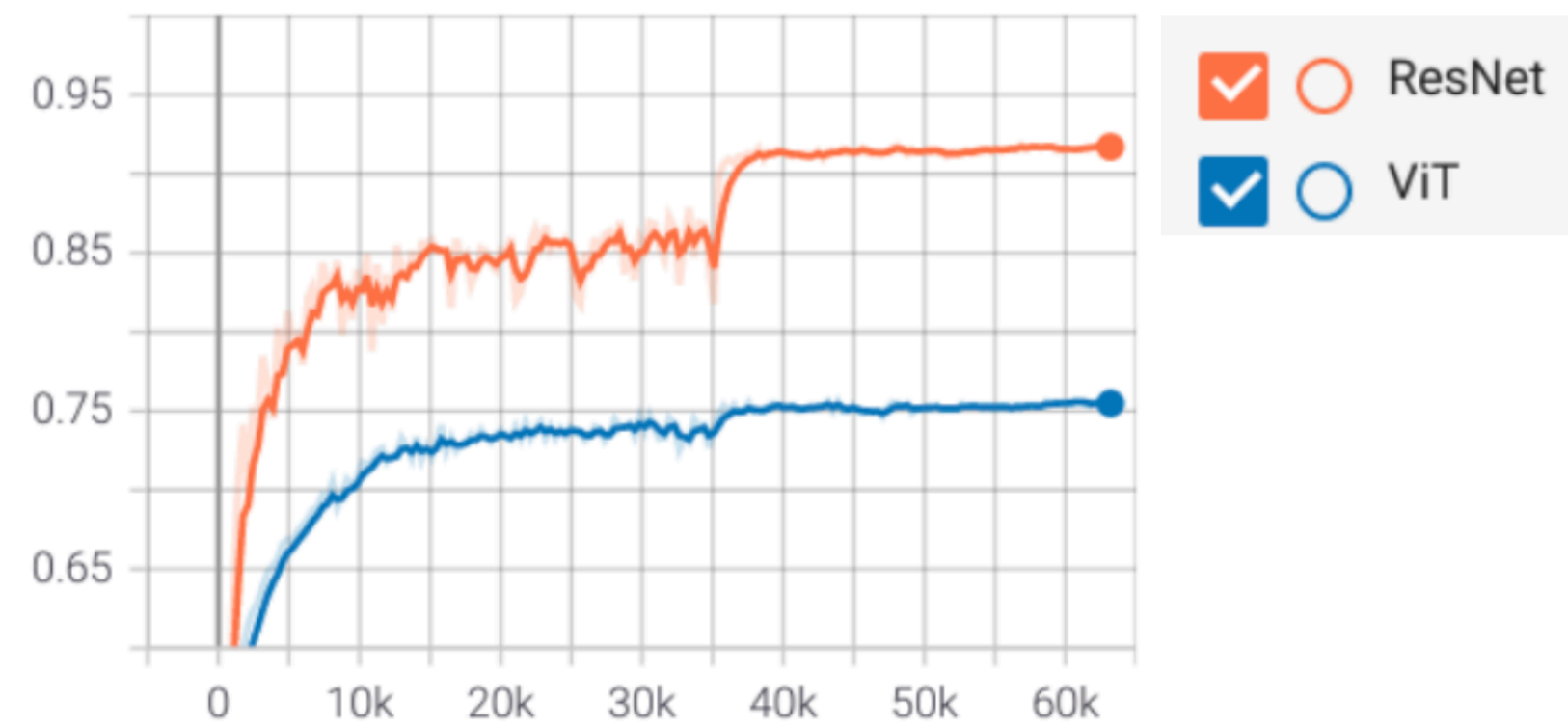
Forward pass of ViT

```
def forward(self, x):  
    # Preprocess input (patchify)  
    x = img_to_patch(x, self.patch_size)  
    B, T, _ = x.shape  
    x = self.input_layer(x)  
    # Add CLS token and positional encoding  
    cls_token = self.cls_token.repeat(B, 1, 1)  
    x = torch.cat([cls_token, x], dim=1)  
    x = x + self.pos_embedding[:, :T+1]  
    # Apply Transformer (with multiple layers)  
    x = self.dropout(x)  
    x = x.transpose(0, 1)  
    x = self.transformer(x)
```



Results

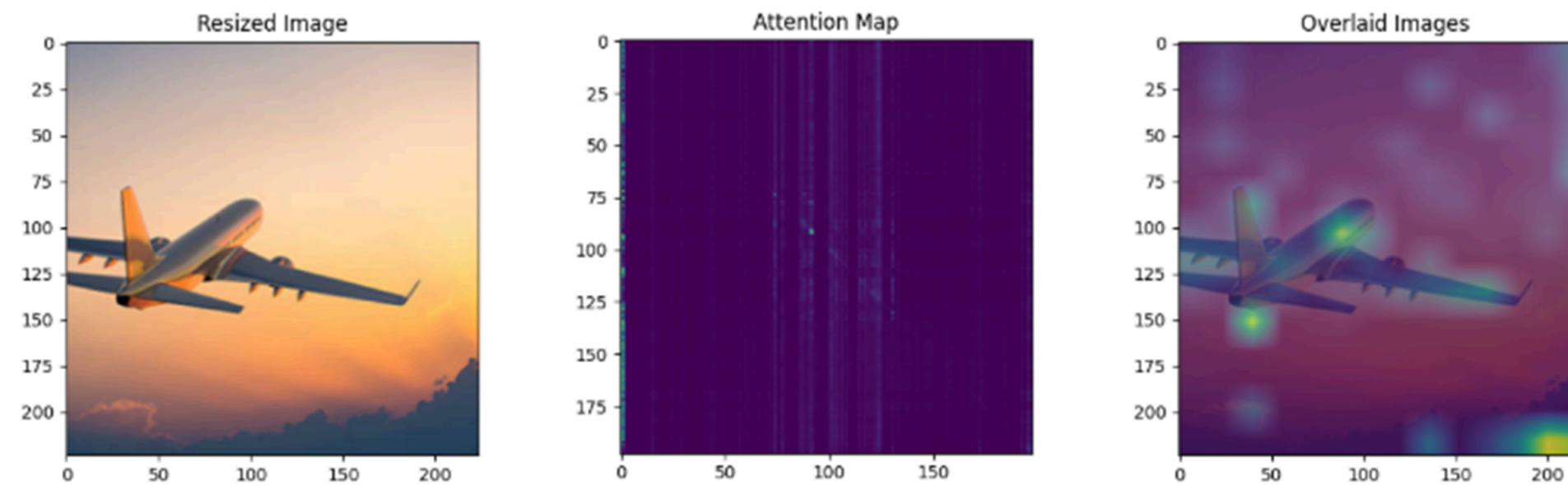
- ResNet outperforms ViT on CIFAR-10
- Inductive biases of CNNs win out if you have limited data/compute
- Transformers have very little inductive bias
 - More flexible, but also more data hungry



Attention visualization

It helps visualizing which parts of an image the model focuses on for the final prediction.

Looking at a single layer is not sufficient , but we need to aggregate the attention maps from multiple layers to have a comprehensive view, which is called **attention rollout**.



Step 1: Average the attention maps from all heads (h) of each layer (l).

$$\bar{A}^l = \frac{1}{H} \sum_h A^{hl}$$

Step 2: Add the identity matrix (I) to correct for the residual connection.

$$\hat{A}^l = 0.5\bar{A}^l + 0.5I,$$

Step 3: For the actual rollout we multiply the attention matrices from each layer sequentially.

$$R = \prod_{i=1}^L \hat{A}^i$$

Step 4: We then look at the attention rollout corresponding to the [CLS] token.

```

# --- Compute Rollout ---
# Start with the attention matrix of the first layer
rollout = attentions[0]

# Iterate through the remaining layers
for i in range(1, len(attendings)):
    attn = attentions[i]

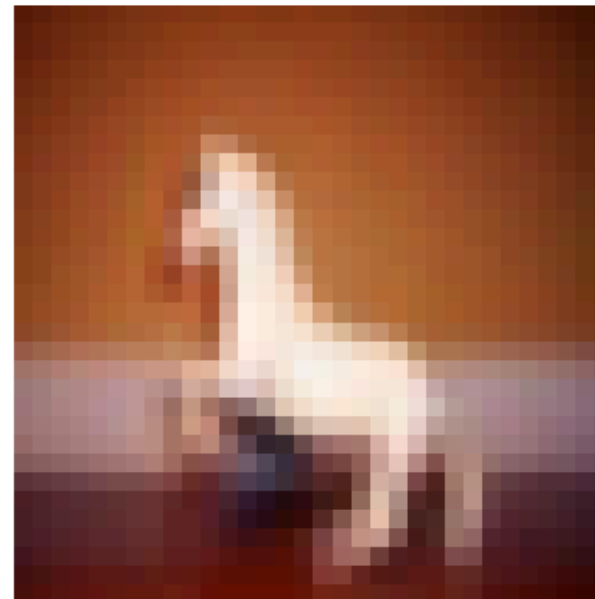
    # Step 2: Add the residual connection (Identity matrix)
    # The formula is: 0.5 * I + 0.5 * Attention
    id_matrix = torch.eye(rollout.shape[0])
    attn_residual = 0.5 * torch.eye(rollout.shape[0]) + 0.5 * attn

    # Step 3: Multiply
    rollout = torch.matmul(attn_residual, rollout)

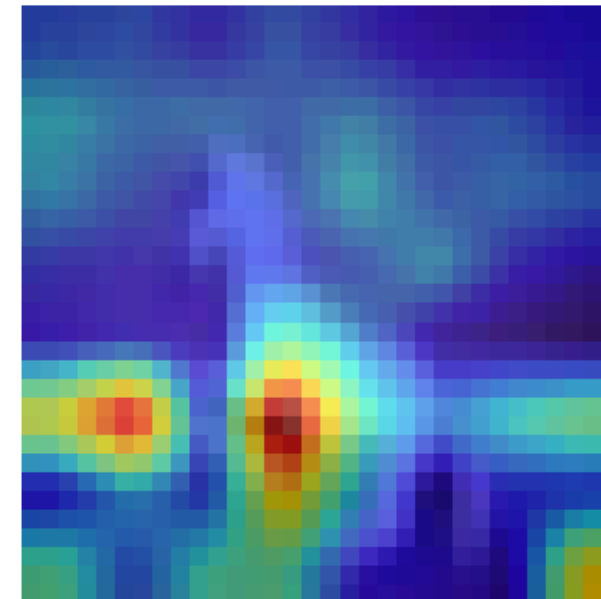
# Extract the attention of the CLS token (row 0) to all patches
cls_attention = rollout[0]
# Discard the CLS token's attention to itself (index 0) to get only patch attention
cls_attention = cls_attention[1:]

```

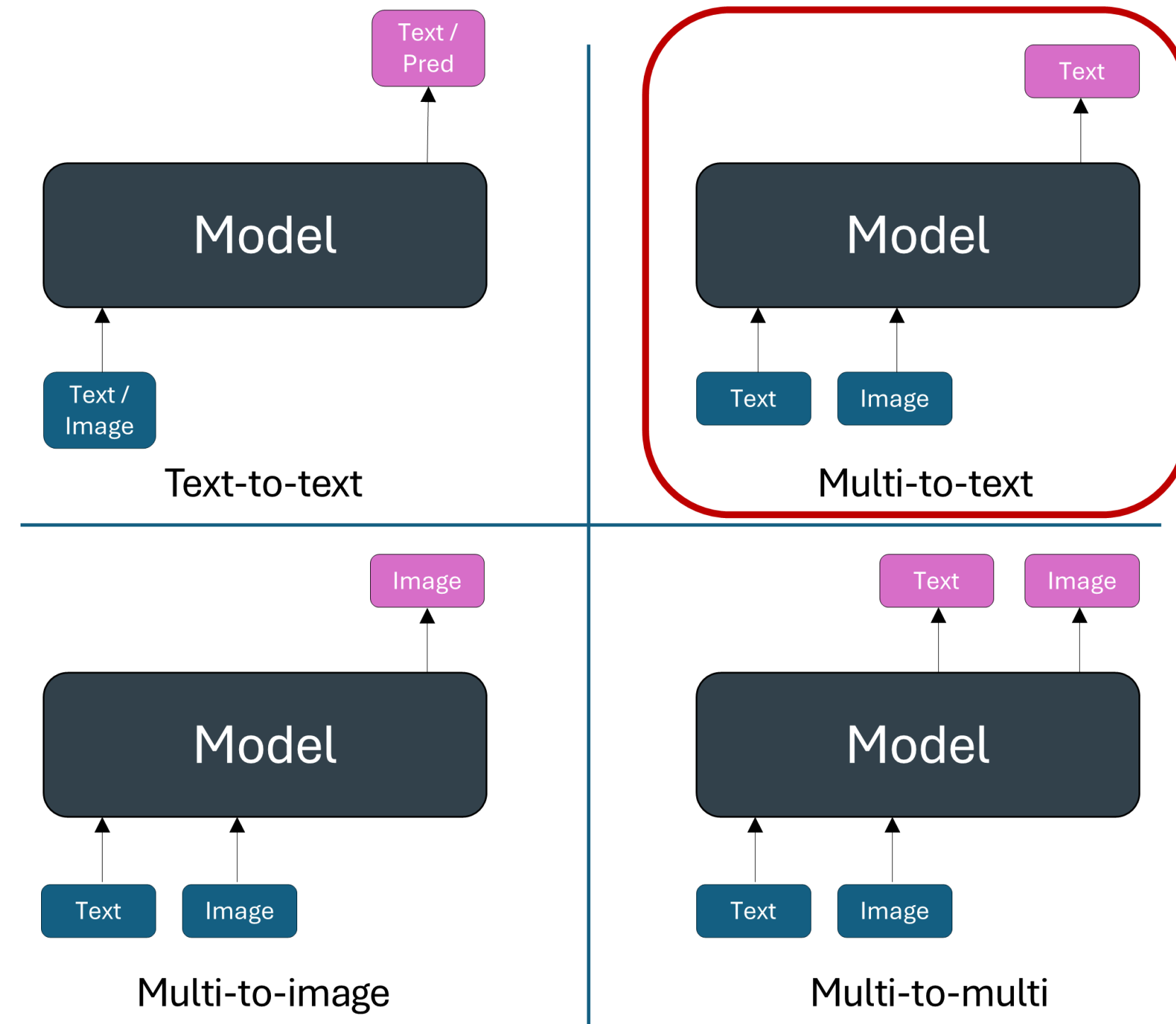
Original Image



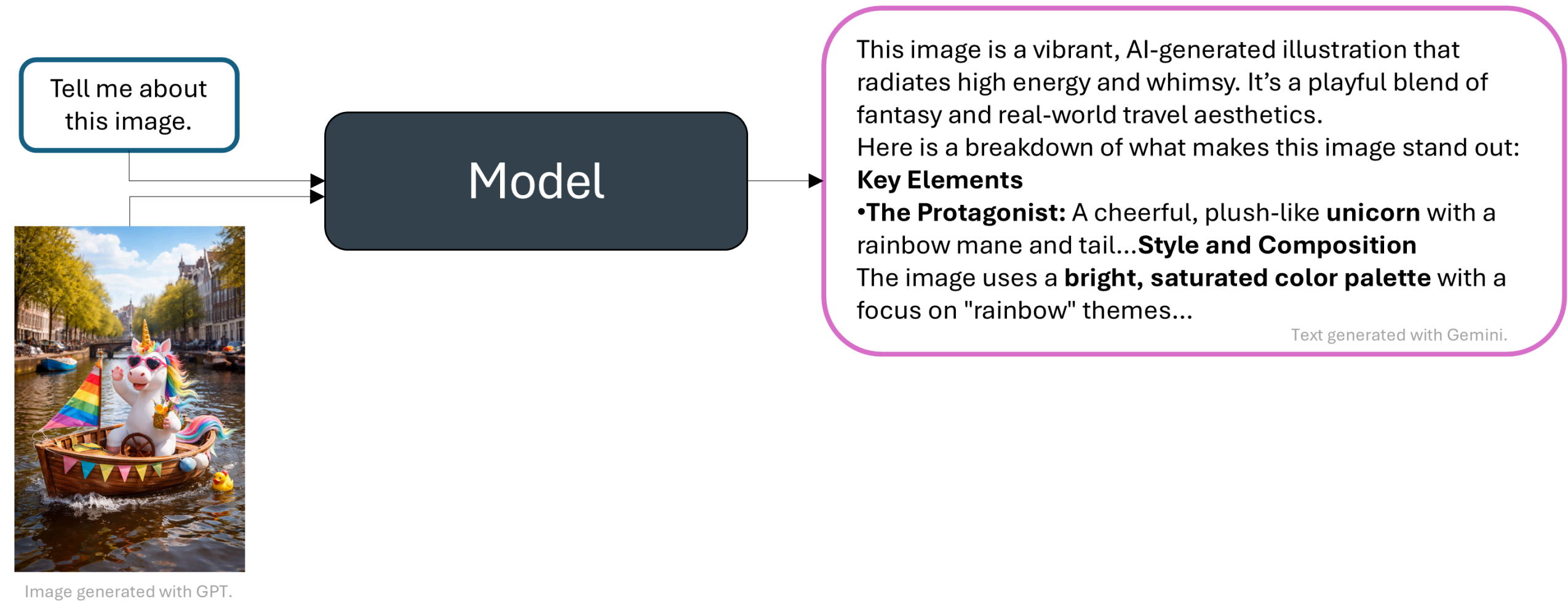
Attention Rollout



Multimodal models with vision and text



Let's look at an example.



And there are many more, for instance mathematical reasoning with an image plot, web agents with webpage screenshot, etc.

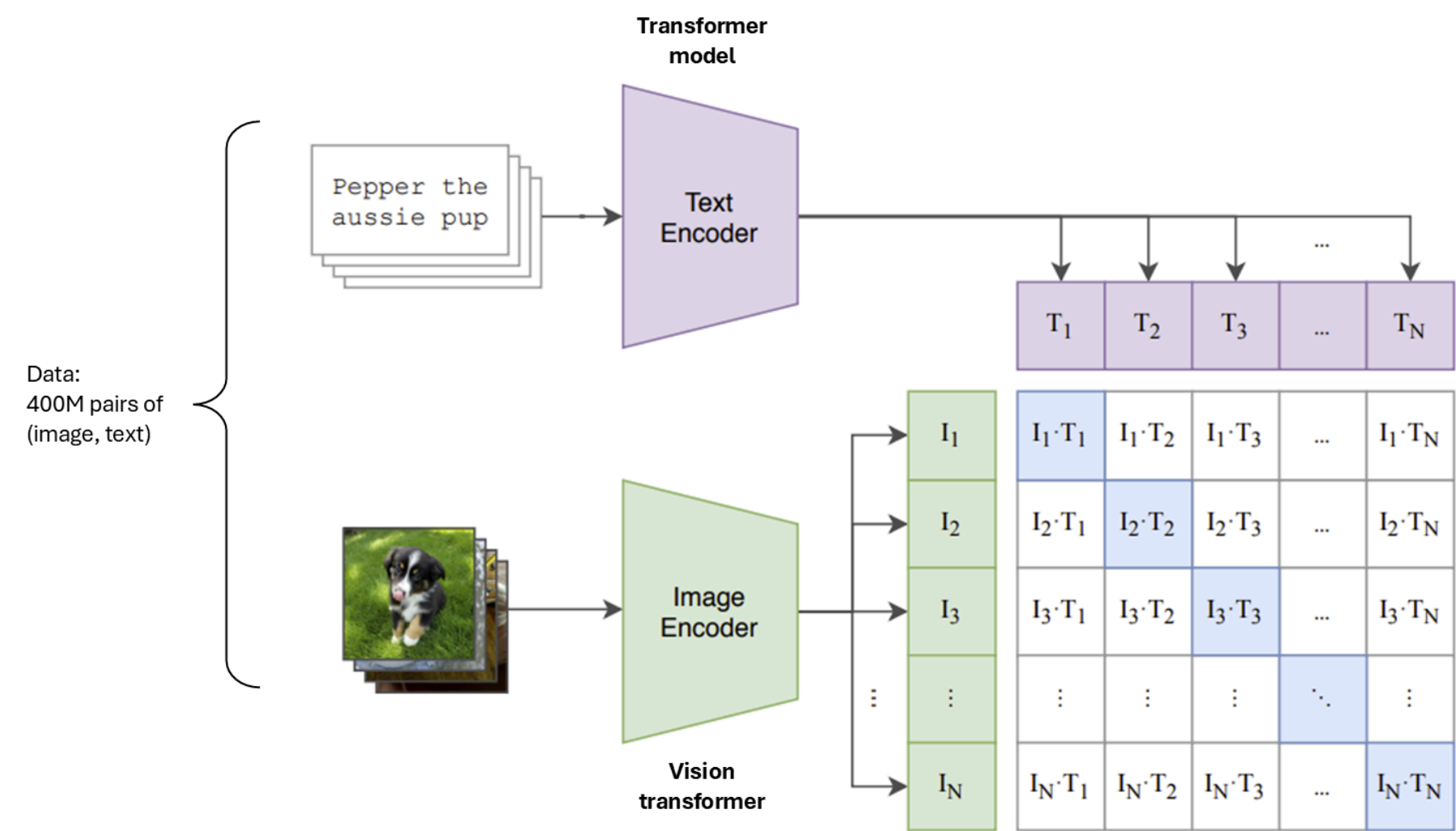
Contrastive Language-Image Pre-Training (CLIP)

Goal: training a model to learn a good image representation.

Idea: Learn from images and text jointly without label supervision.

- Learn an image encoder $f_I(x_I) \rightarrow z_I$.
- Learn a text encoder $f_T(x_T) \rightarrow z_T$.
- Force the representations for an image and corresponding text to be close together.
- Force the representations for an image and an unpaired text to be far apart.

Given N (image, text) pairs, classify which image is paired with which text.



Source: [Learning Transferable Visual Models From Natural Language Supervision](#) (Radford et al.)

Training loss for CLIP

$$L((x_1, y_1), \dots, (x_N, y_N)) =$$
$$= -\frac{1}{2} \sum_{n=1}^N \left[\log \frac{\exp(f_I(x_n)^T f_T(y_n))}{\sum_j \exp(f_I(x_j)^T f_T(y_n))} + \log \frac{\exp(f_I(x_n)^T f_T(y_n))}{\sum_j \exp(f_I(x_n)^T f_T(y_j))} \right]$$

Push up the dot product of the correct pair

Push down the dot product of the other pairs

Pseudocode for training loss

```
# image_encoder - ResNet or Vision Transformer
# text_encoder  - CBOW or Text Transformer
# I[n, h, w, c] - minibatch of aligned images
# T[n, l]       - minibatch of aligned texts
# W_i[d_i, d_e] - learned proj of image to embed
# W_t[d_t, d_e] - learned proj of text to embed
# t             - learned temperature parameter

# extract feature representations of each modality
I_f = image_encoder(I) #[n, d_i]
T_f = text_encoder(T)  #[n, d_t]

# joint multimodal embedding [n, d_e]
I_e = l2_normalize(np.dot(I_f, W_i), axis=1)
T_e = l2_normalize(np.dot(T_f, W_t), axis=1)

# scaled pairwise cosine similarities [n, n]
logits = np.dot(I_e, T_e.T) * np.exp(t)

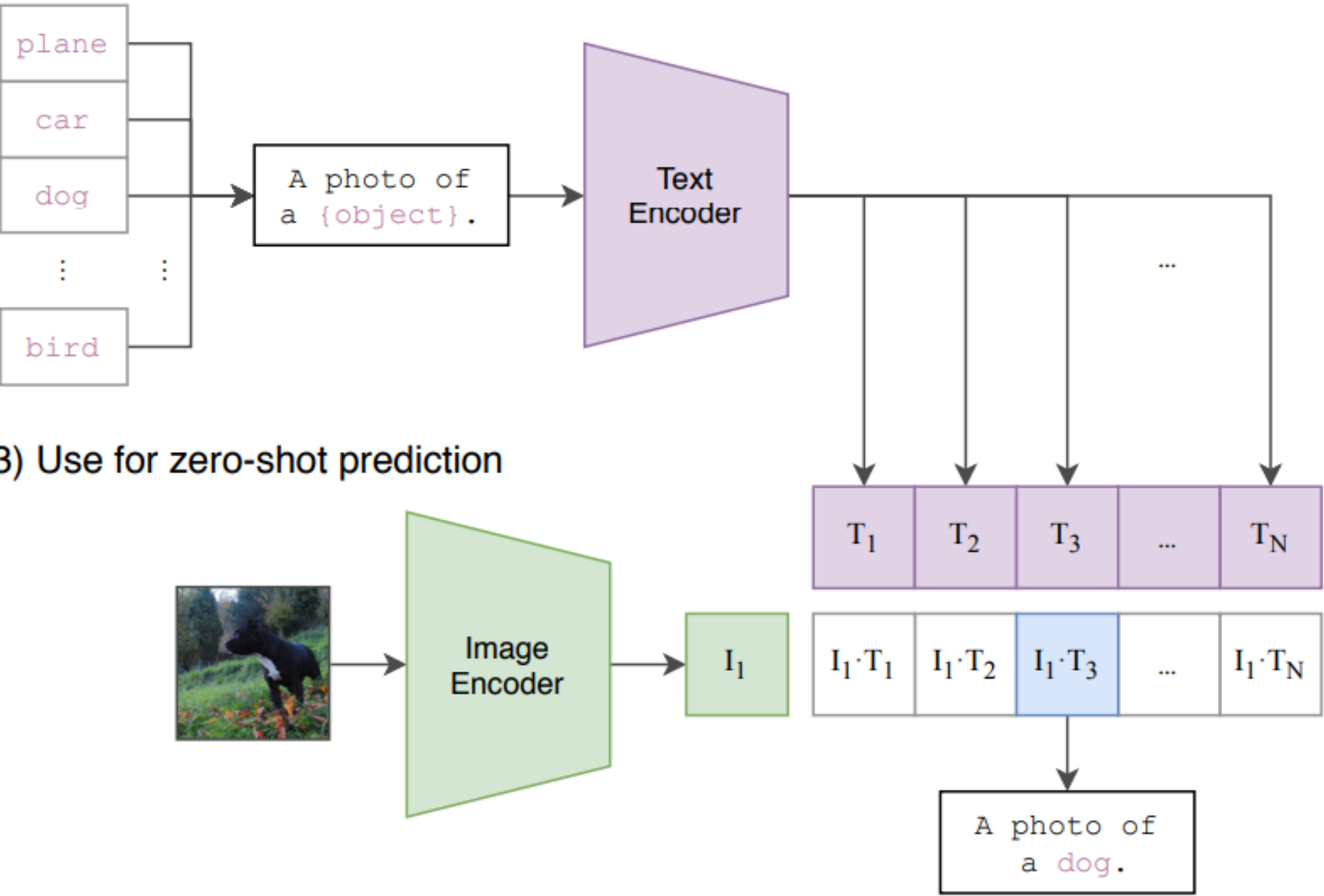
# symmetric loss function
labels = np.arange(n)
loss_i = cross_entropy_loss(logits, labels, axis=0)
loss_t = cross_entropy_loss(logits, labels, axis=1)
loss   = (loss_i + loss_t)/2
```

Source: Learning Transferable Visual Models From Natural Language Supervision (Radford et al.)

Zero-shot classification

After the training phase you end up with a good pre-trained text and image encoder and you can use them for zero-shot classification (without any additional examples).

(2) Create dataset classifier from label text



Source: [Learning Transferable Visual Models From Natural Language Supervision \(Radford et al.\)](#)

Let's look at the Python code!

```
from transformers import CLIPProcessor, CLIPModel

# Load model
model = CLIPModel.from_pretrained("openai/clip-vit-base-patch32")
processor = CLIPProcessor.from_pretrained("openai/clip-vit-base-patch32") # wraps an image processor and a tokenizer
print("CLIP model loaded successfully!")

# Load image from COCO dataset
url = "https://farm1.staticflickr.com/166/343915369_895049d44f_z.jpg"
image = Image.open(requests.get(url, stream=True).raw)
```




```
# Define text descriptions
texts = ["a photo of a giraffe", "a photo of a horse"]

# Process inputs
inputs = processor(text=texts, images=image, return_tensors="pt", padding=True)

# Get model outputs
outputs = model(**inputs)
logits_per_image = outputs.logits_per_image # image-text similarity scores
probs = logits_per_image.softmax(dim=1) # convert to probabilities

# Display results
print("Giraffe vs Horse Classification")

# Most likely
max_idx = probs[0].argmax().item()
print(f"\nPrediction: {texts[max_idx]} ({probs[0][max_idx].item():.1%})")
```

Giraffe vs Horse Classification

Raw similarity scores (logits):

a photo of a giraffe: 29.609

a photo of a horse: 22.280

Probabilities:

a photo of a giraffe: 99.9%

a photo of a horse: 0.1%

Prediction: a photo of a giraffe (99.9%)

Llava: Large Language and Vision Assistant

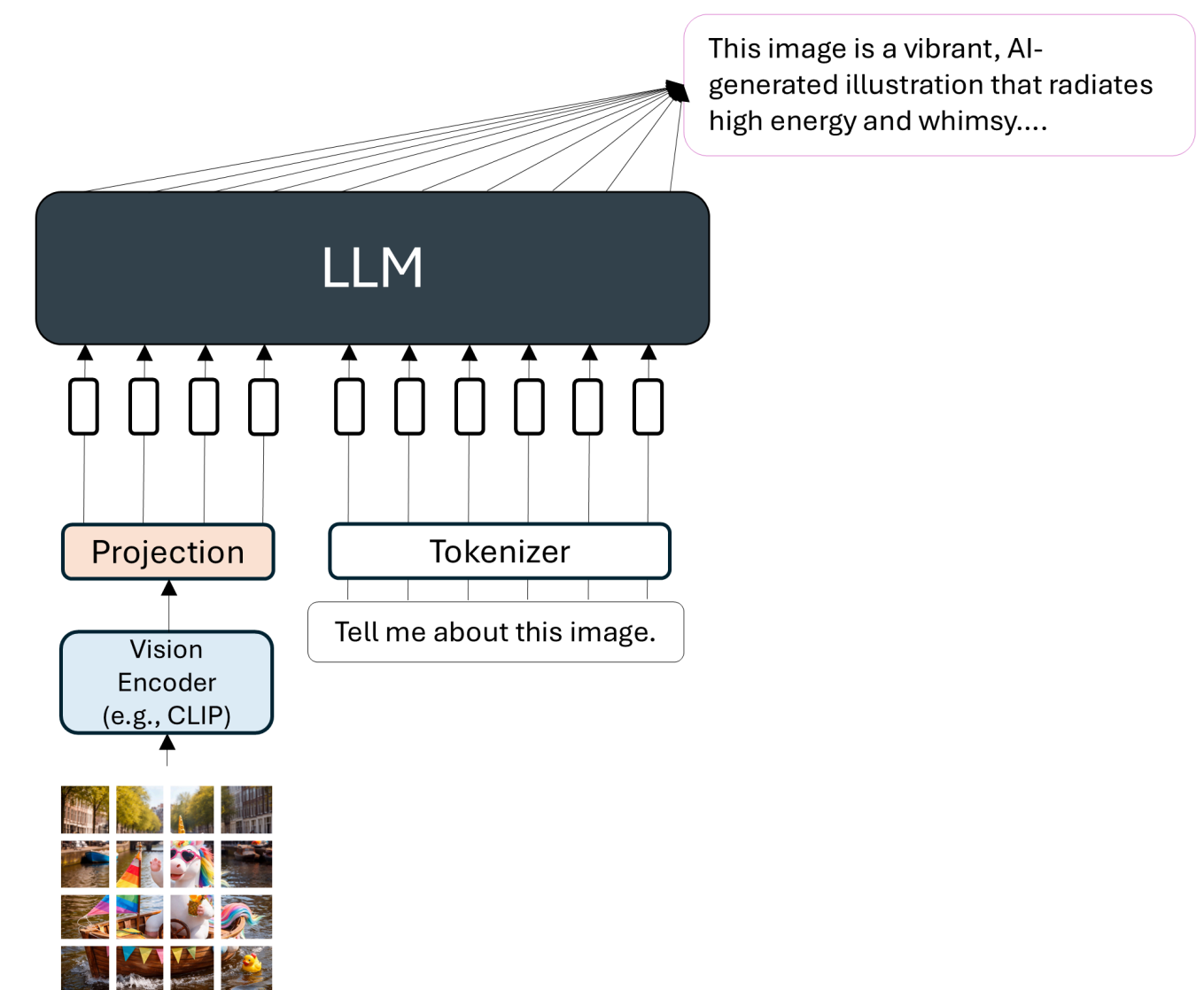
A language model (LLM) that takes in input images and text. The idea is to project the "image tokens" to the embedding space of the text tokens and fine-tuning the model with image-text data.

Step 1: **Image preprocessing** by splitting the image into patches and vectorize them.

Step 2: **Image encoding** using a pre-trained transformer model (e.g., CLIP) and extracting the vectors from the last layer.

Step 3: **Pass the encodings to the LLM** after projecting them in the space of the text tokens, with a simple MLP.

Step 4: **Train/Fine-tune** on data with text and images. The objective is always next-token predictions computed on the output text tokens.

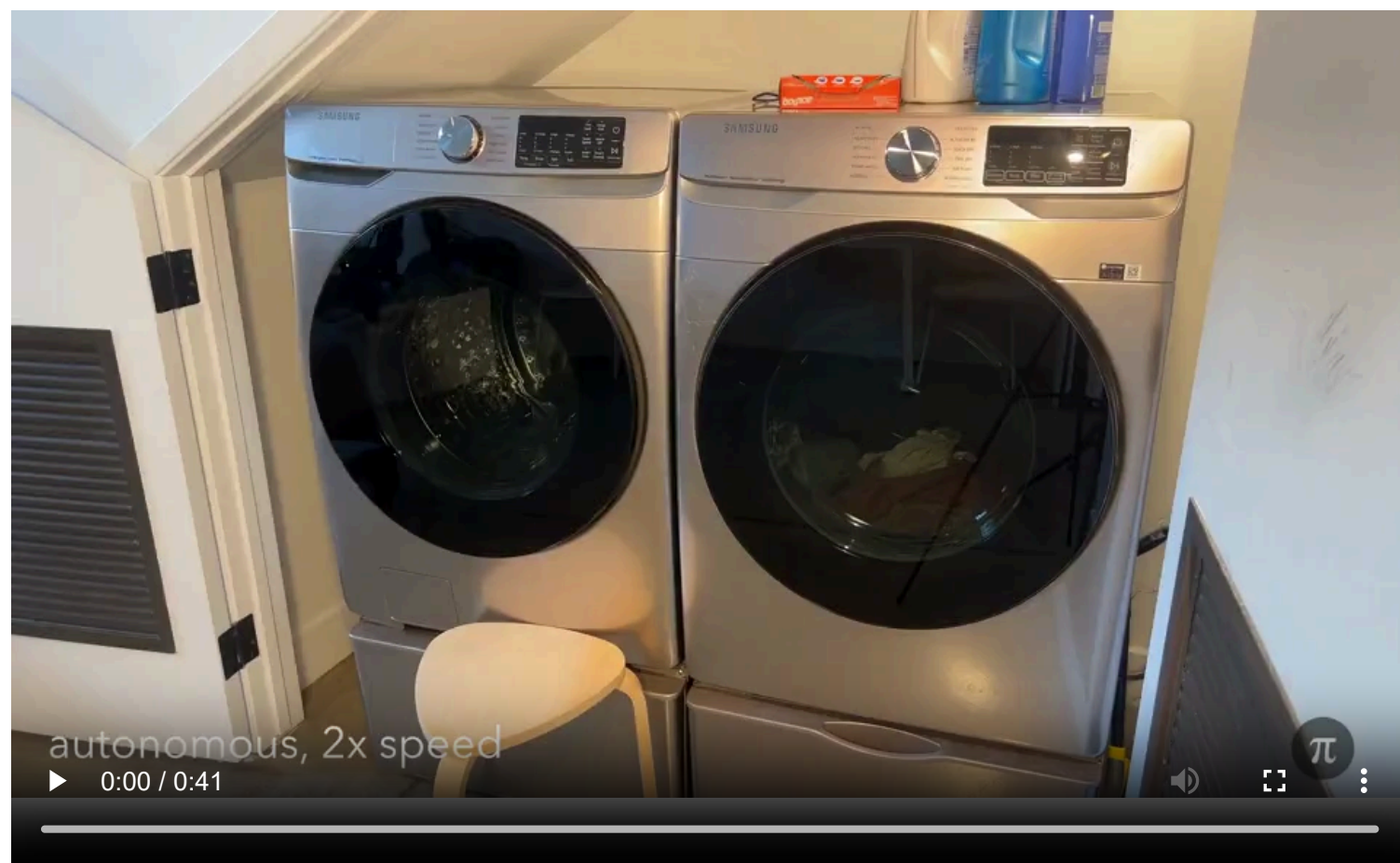


Visual input example, Extreme Ironing:	
	
	Source: https://www.barnorama.com/wp-content/uploads/2016/12/03-Confusing-Pictures.jpg
User	What is unusual about this image?
LLaVA	The unusual aspect of this image is a man ironing clothes on the back of a minivan or van. This is not a typical place to perform this activity, as one would usually iron clothes in a more stationary and safe location, such as a home, using a regular ironing board. The scene depicted in the image is peculiar as it involves a makeshift ironing setup on a vehicle, which can be both unsafe and unconventional. Additionally, it is not clear how the man is able to maintain balance and stability while ironing clothes in such an unstable environment.
User	What is unusual about this image?
GPT-4 [36]	The unusual thing about this image is that a man is ironing clothes on an ironing board attached to the roof of a moving taxi.
User	What is unusual about this image?
BLIP-2	a man is sitting on the back of a yellow cab

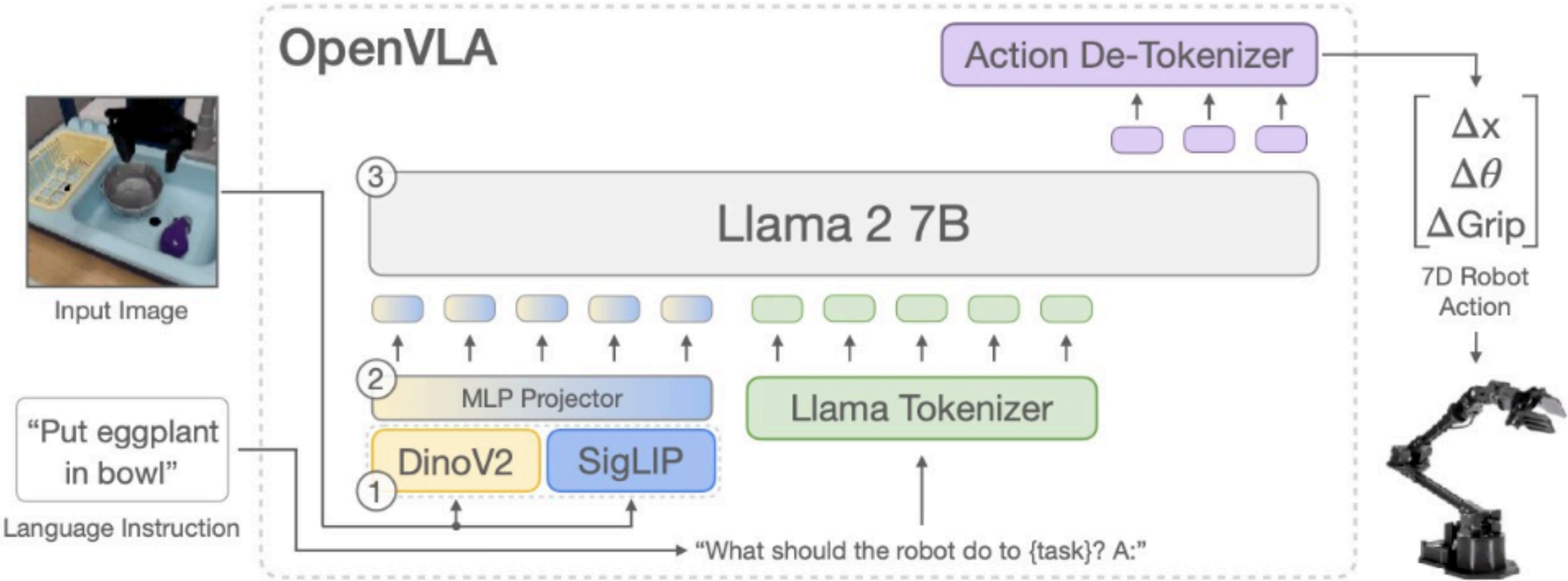
Text generation models with other modalities

```
In [2]: from IPython.display import Video  
  
Video("images/pi0.mp4", width=750, height=500)
```

Out[2]:

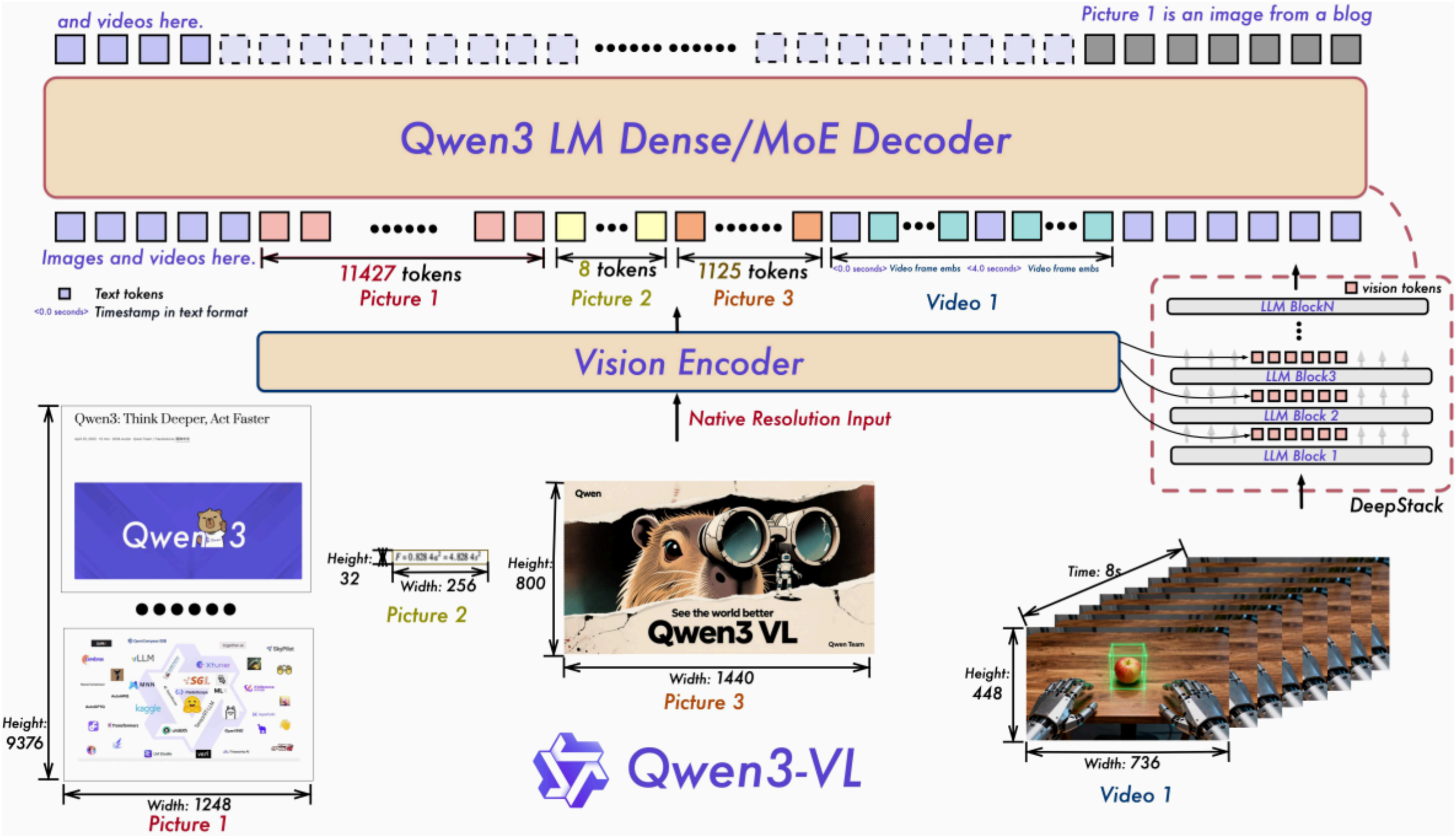


Vision-Language Action Models (VLAs)



Source: [OpenVLA \(Kim et al.\)](#)

Video-Language Models (VLMs)



Source: Qwen3-VL Technical Report (Qwen Team)

An example



pi0.mp4

2 MB

What is the robotic arm doing in this video?

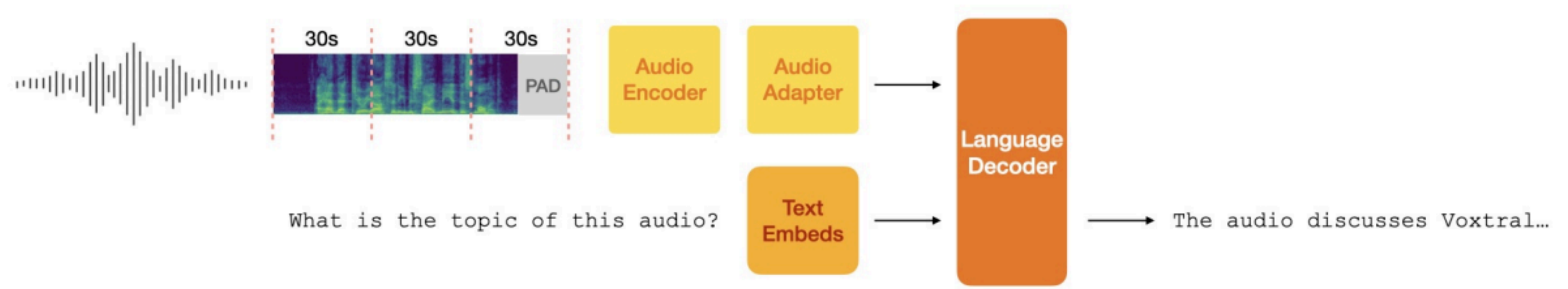


Qwen3-VL

In the video, the robotic arm is performing an autonomous laundry task. It opens the dryer door, reaches inside to grab clothes one by one, and places them into a nearby laundry basket. The process is shown at 2x speed, highlighting the robot's efficiency and precision in handling various garments. After removing all the clothes, the robotic arm closes the dryer door, completing the task. This demonstrates advanced robotics capabilities in household chores.

Source: [Hugging Face space \(Qwen/Qwen3-VL-Demo\)](#)

Audio-Text-to-Text Models



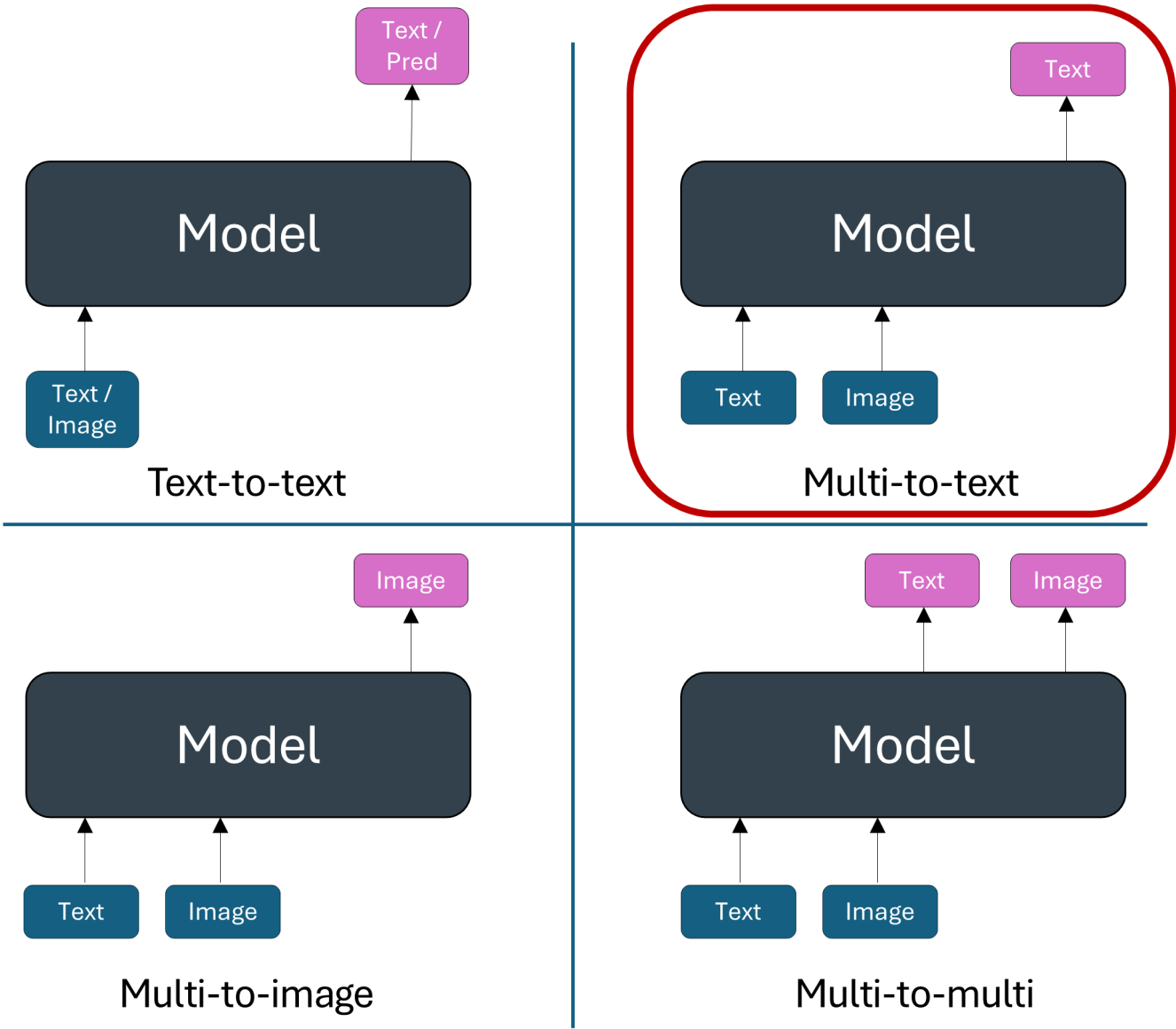
Source: [Voxtral \(Mistral.AI\)](#)

Examples of use cases:

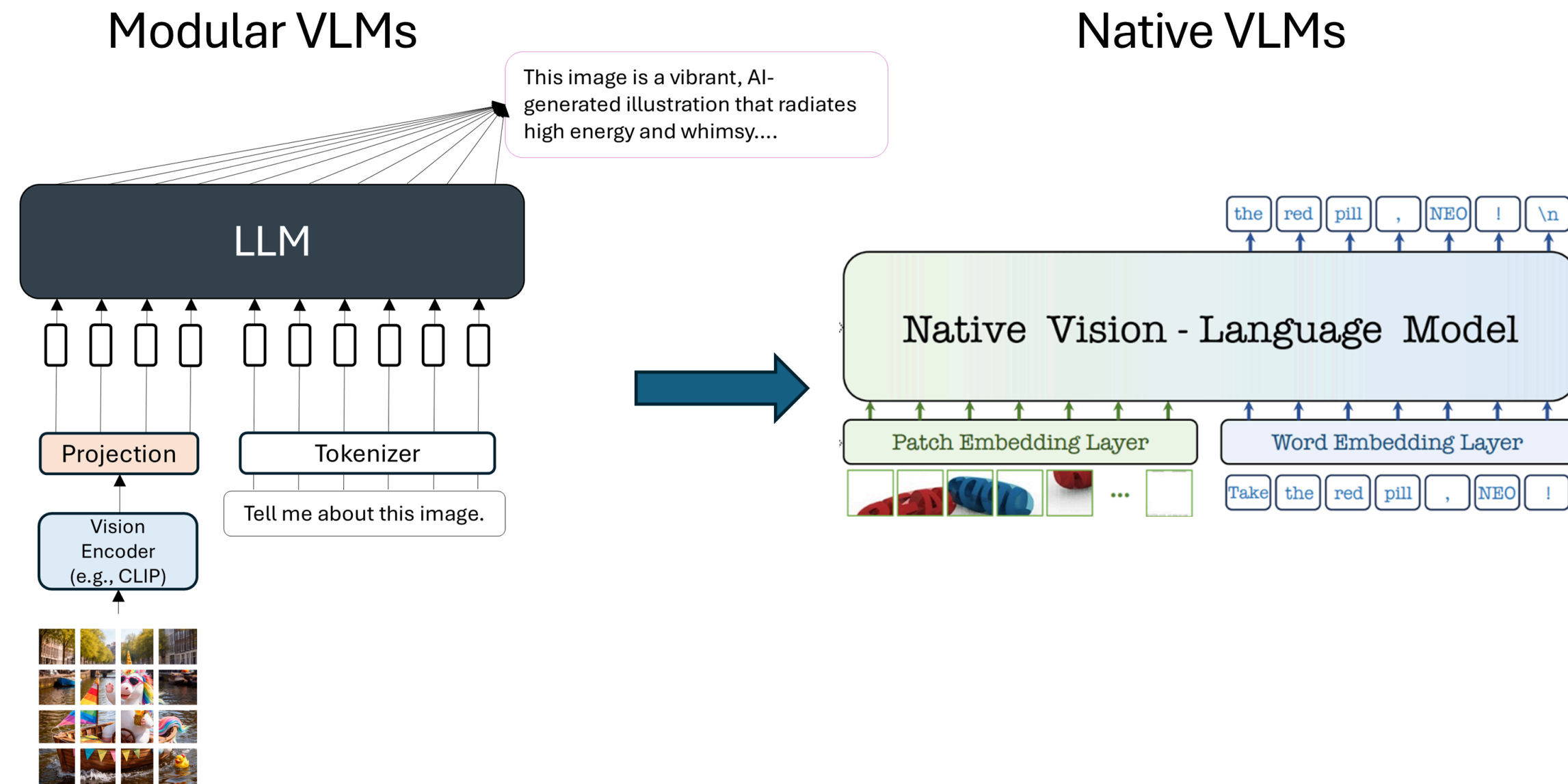
- Audio QA: ask questions about lectures, podcasts, or calls and get context-aware answers.
- Meeting notes & action items: turn multi-speaker meetings into concise minutes with decisions, owners, and deadlines.
- Speech understanding & intent: extract information about the speech and the intent, sentiment, uncertainty, or emotion from spoken language.
- Music & sound analysis: describe instrumentation, genre, tempo, or sections and suggest edits.

And many more, such as tabular-text models, timeseries-text models, etc.

But, remember that we only focused on Multi-to-text in this lecture. More can be done when generating also images, videos, etc.



What's next?



Check the following papers to learn more:

- Team, Chameleon. "Chameleon: Mixed-modal early-fusion foundation models." arXiv preprint arXiv:2405.09818 (2024).
- Sun, Quan, et al. "Emu: Generative pretraining in multimodality." arXiv preprint arXiv:2307.05222 (2023).

Useful material

- Ebrahim Pichka. [What is Query, Key, and Value \(QKV\) in the Transformer Architecture and Why Are They Used?](#) (2023).
- Sebastian Raschka. [Understanding Multimodal LLMs](#). (November 2024)
- Stanford CS231n course. [Deep Learning for Computer Vision](#). (Spring 2025).
- CMU course. [Advanced Natural Language Processing](#). (Spring 2025).
- Samira Abnar. [Quantifying Attention Flow in Transformers](#). (Spring 2020).
- Jacob Gildenblat. [Exploring Explainability for Vision Transformers](#)
- Liu, Haotian, et al. "Visual instruction tuning." Advances in neural information processing systems 36 (2023).
- Bai, Shuai, et al. "Qwen3-vl technical report." arXiv preprint arXiv:2511.21631 (2025).
- Liu, Alexander H., et al. "Voxtral." arXiv preprint arXiv:2507.13264 (2025).

